

CSS Grid Items Positioning

Easy Guide with Examples

Contents

1	Introduction to Grid Items Positioning	2
1.1	Why Grid Items Positioning Matters	2
1.2	Key Concepts	2
2	Understanding Grid Lines	3
2.1	How Grid Lines Work	3
2.2	Line Numbering Example	3
2.3	Negative Line Numbers	3
3	grid-column Property	4
3.1	Basic Syntax	4
3.2	Example 1: Basic Column Positioning	4
3.3	Example 2: Using Negative Lines	4
3.4	Example 3: Using span	5
4	grid-row Property	6
4.1	Basic Syntax	6
4.2	Example 1: Basic Row Positioning	6
4.3	Example 2: Row with Span	6
4.4	Example 3: Positioning at the End	7
5	grid-area Property - Shorthand	8
5.1	Syntax	8
5.2	Example 1: Simple Positioning	8
5.3	Example 2: Using Span	8
5.4	Comparison: Which Should You Use?	8
6	Auto-Placement Algorithm	10
6.1	How Auto-Placement Works	10
6.2	Controlling Auto-Placement Direction	10
6.3	Example: Dense Packing	11
7	Practical Examples	12
7.1	Example 1: Magazine Layout	12
7.2	Example 2: Dashboard Layout	13
7.3	Example 3: Overlapping Items	13

8	Best Practices	15
8.1	Do's	15
8.2	Don'ts	15
9	Common Mistakes	16
9.1	Mistake 1: Wrong Line Numbers	16
9.2	Mistake 2: Forgetting the Slash	16
9.3	Mistake 3: Confusing Span Syntax	16
9.4	Mistake 4: Overlapping Items	17
10	Advanced Positioning Techniques	18
10.1	Using justify-self for Horizontal Alignment	18
10.2	Using align-self for Vertical Alignment	18
10.3	Combining Positioning with Alignment	18
10.4	Dynamic Positioning with CSS Variables	19
11	Responsive Grid Positioning	20
11.1	Mobile-First Approach	20
11.2	Desktop-First Approach	20
12	Practice Exercises	22
12.1	Exercise 1: Create a 2x2 Grid	22
12.2	Exercise 2: Magazine Layout	22
12.3	Exercise 3: Dashboard	22
12.4	Exercise 4: Responsive Navigation	23
13	Summary	24
13.1	Key Takeaways	24
13.2	Quick Reference	24
13.3	Next Steps	25

Chapter 1

Introduction to Grid Items Positioning

CSS Grid is a powerful 2D layout system that allows you to precisely position items on a grid. Grid Items Positioning refers to how individual elements (items) are placed within the grid container using various CSS properties.

Unlike Flexbox which works in one dimension, CSS Grid lets you control both rows and columns simultaneously. This chapter will teach you how to position items exactly where you want them on the grid.

1.1 Why Grid Items Positioning Matters

- **Precise Control:** Position any item anywhere on the grid
- **Complex Layouts:** Create sophisticated multi-column/row layouts easily
- **Responsive Design:** Reposition items without changing HTML
- **Overlap:** Allow items to overlap for creative designs
- **Alignment:** Control alignment within grid cells

1.2 Key Concepts

Grid Lines: Invisible lines that form rows and columns **Grid Cells:** The space between grid lines

Grid Track: A row or column of the grid **Grid Item:** Any child element of the grid container

Chapter 2

Understanding Grid Lines

Before positioning items, you must understand grid lines. They are numbered starting from 1.

2.1 How Grid Lines Work

If you have a 3-column grid:

```
Grid with 3 columns has 4 vertical lines:  
| 1 | 2 | 3 | 4 |  
Col1 Col2 Col3
```

Similarly, 3 rows have 4 horizontal lines.

2.2 Line Numbering Example

```
.container {  
  display: grid;  
  grid-template-columns: 100px 100px 100px;  
  grid-template-rows: 50px 50px;  
}
```

This creates:

- 3 columns and 2 rows
- Vertical lines numbered: 1, 2, 3, 4
- Horizontal lines numbered: 1, 2, 3

2.3 Negative Line Numbers

Grid lines can also be counted from the end using negative numbers:

- Last line = -1
- Second to last = -2
- And so on...

Chapter 3

grid-column Property

The `grid-column` property positions an item horizontally using column lines.

3.1 Basic Syntax

```
grid-column: start / end;
```

start: The starting column line **end:** The ending column line (exclusive)

3.2 Example 1: Basic Column Positioning

```
HTML:
<div class="container">
  <div class="item">Item 1</div>
  <div class="item">Item 2</div>
  <div class="item">Item 3</div>
</div>

CSS:
.container {
  display: grid;
  grid-template-columns: 100px 100px 100px;
  gap: 10px;
}

.item:first-child {
  grid-column: 1 / 3; /* Spans from line 1 to 3 */
}

.item:nth-child(2) {
  grid-column: 3 / 4; /* Spans from line 3 to 4 */
}
```

Result: First item takes 2 columns, second item takes 1 column

3.3 Example 2: Using Negative Lines

```
.item {  
  grid-column: 1 / -1; /* Spans entire width */  
}
```

3.4 Example 3: Using span

```
.item:first-child {  
  grid-column: 1 / span 2; /* Start at line 1, span 2 columns */  
}  
  
.item:nth-child(2) {  
  grid-column: span 1; /* Span 1 column from current position */  
}
```

Chapter 4

grid-row Property

The `grid-row` property positions an item vertically using row lines.

4.1 Basic Syntax

```
grid-row: start / end;
```

4.2 Example 1: Basic Row Positioning

```
HTML:
<div class="container">
  <div class="item">Item 1</div>
  <div class="item">Item 2</div>
  <div class="item">Item 3</div>
</div>

CSS:
.container {
  display: grid;
  grid-template-columns: 100px 100px;
  grid-template-rows: 50px 50px 50px;
  gap: 10px;
}

.item:first-child {
  grid-row: 1 / 3; /* Spans from row line 1 to 3 */
}

.item:nth-child(2) {
  grid-row: 2 / 4;
}
```

4.3 Example 2: Row with Span


```
.item:first-child {  
  grid-row: 1 / span 2; /* Start at row 1, span 2 rows */  
}  
  
.item:nth-child(2) {  
  grid-row: span 1; /* Span only 1 row */  
}
```

4.4 Example 3: Positioning at the End

```
.item {  
  grid-row: 3 / -1; /* From row 3 to the last row */  
}
```

Chapter 5

grid-area Property - Shorthand

The `grid-area` property combines `grid-row` and `grid-column` in one property.

5.1 Syntax

```
grid-area: row-start / column-start / row-end / column-end;
```

Order matters! Remember: **ROW-START / COLUMN-START / ROW-END / COLUMN-END**

5.2 Example 1: Simple Positioning

```
.item:first-child {  
  grid-area: 1 / 1 / 3 / 3; /* Row 1-3, Column 1-3 */  
}  
  
.item:nth-child(2) {  
  grid-area: 2 / 3 / 3 / 4; /* Row 2-3, Column 3-4 */  
}
```

5.3 Example 2: Using Span

```
.item:first-child {  
  grid-area: 1 / 1 / span 2 / span 2;  
  /* Start at (1,1), span 2 rows and 2 columns */  
}  
  
.item:nth-child(2) {  
  grid-area: 1 / 3 / span 1 / span 1;  
}
```

5.4 Comparison: Which Should You Use?

- Use `grid-column` when you only need horizontal positioning

- Use **grid-row** when you only need vertical positioning
- Use **grid-area** when you need to position both row and column together

Chapter 6

Auto-Placement Algorithm

When you don't specify positions, CSS Grid automatically places items using the auto-placement algorithm.

6.1 How Auto-Placement Works

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 100px);  
  grid-template-rows: repeat(2, 50px);  
}  
  
/* No positioning specified - items auto-place */  
.item:nth-child(1) { /* Grid cell 1 */ }  
.item:nth-child(2) { /* Grid cell 2 */ }  
.item:nth-child(3) { /* Grid cell 3 */ }
```

Items fill left-to-right, top-to-bottom by default.

6.2 Controlling Auto-Placement Direction

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 100px);  
  grid-auto-flow: row; /* Default: left-to-right */  
}  
  
.container.column {  
  grid-auto-flow: column; /* Top-to-bottom, then next column */  
}  
  
.container.dense {  
  grid-auto-flow: row dense; /* Fill empty spaces */  
}
```

6.3 Example: Dense Packing

```
HTML:
<div class="container">
  <div class="item wide">Wide Item</div>
  <div class="item">A</div>
  <div class="item">B</div>
</div>

CSS:
.container {
  display: grid;
  grid-template-columns: repeat(4, 100px);
  grid-auto-flow: row dense;
}

.wide {
  grid-column: span 2;
}

/* Without 'dense': B would be on row 2
   With 'dense': B fills the space left by Wide Item */
```

Chapter 7

Practical Examples

7.1 Example 1: Magazine Layout

```
HTML:
<div class="magazine">
  <header>Header</header>
  <main>Main Content</main>
  <aside>Sidebar</aside>
  <footer>Footer</footer>
</div>

CSS:
.magazine {
  display: grid;
  grid-template-columns: 1fr 250px;
  grid-template-rows: auto 1fr auto;
  gap: 20px;
  height: 100vh;
}

header {
  grid-column: 1 / -1; /* Span all columns */
  grid-row: 1;
  background: #333;
  padding: 20px;
}

main {
  grid-column: 1;
  grid-row: 2;
}

aside {
  grid-column: 2;
  grid-row: 2;
  background: #f5f5f5;
}

footer {
```

```
grid-column: 1 / -1;
grid-row: 3;
background: #333;
padding: 20px;
}
```

7.2 Example 2: Dashboard Layout

```
HTML:
<div class="dashboard">
  <div class="widget big">Widget 1</div>
  <div class="widget">Widget 2</div>
  <div class="widget">Widget 3</div>
  <div class="widget">Widget 4</div>
</div>

CSS:
.dashboard {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-template-rows: repeat(2, 200px);
  gap: 15px;
}

.widget.big {
  grid-column: span 2;
  grid-row: span 2;
  background: #4CAF50;
}

.widget {
  background: #2196F3;
}
```

7.3 Example 3: Overlapping Items

```
HTML:
<div class="container">
  <div class="background">Background</div>
  <div class="content">Content</div>
</div>

CSS:
.container {
  display: grid;
  grid-template-columns: 200px;
  grid-template-rows: 200px;
}

.background {
```

```
    grid-column: 1;
    grid-row: 1;
    background: url('bg.jpg');
}

.content {
    grid-column: 1;
    grid-row: 1;
    background: rgba(0,0,0,0.5);
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 1;
}
```


Chapter 8

Best Practices

8.1 Do's

- **Plan your grid first:** Sketch the layout before writing code
- **Use named areas:** Makes code more readable for complex layouts
- **Use span wisely:** It's easier to read than calculating line numbers
- **Test responsiveness:** Change grid template in media queries, not individual item positions
- **Use auto for flexibility:** Let items auto-place when possible

8.2 Don'ts

- **Don't over-specify:** Only position items that need specific placement
- **Don't mix auto-flow directions:** Stick to one direction for consistency
- **Don't create too many explicit rows:** Use auto-rows/auto-columns instead
- **Don't forget gaps:** They're essential for visual breathing room
- **Don't hardcode sizes:** Use fractions (fr) and repeat() for flexibility

Chapter 9

Common Mistakes

9.1 Mistake 1: Wrong Line Numbers

```
/* WRONG - thinking in terms of columns, not lines */
.item {
  grid-column: 1 / 2; /* Only 1 column, not 2 */
}

/* CORRECT */
.item {
  grid-column: 1 / 3; /* Spans columns 1 and 2 */
}
```

9.2 Mistake 2: Forgetting the Slash

```
/* WRONG */
.item {
  grid-column: 1 3; /* Missing slash - won't work */
}

/* CORRECT */
.item {
  grid-column: 1 / 3;
}
```

9.3 Mistake 3: Confusing Span Syntax

```
/* WRONG */
.item {
  grid-column: 1 / span 3; /* Ambiguous */
}

/* CORRECT - One of these */
.item {
  grid-column: 1 / 4; /* Lines 1 to 4 = 3 columns */
}
```

```
}  
  
.item {  
  grid-column: span 3; /* Span 3 columns */  
}
```

9.4 Mistake 4: Overlapping Items

```
/* Can cause issues if not intentional */  
.item1 {  
  grid-column: 1 / 3;  
  grid-row: 1 / 3;  
}  
  
.item2 {  
  grid-column: 2 / 4;  
  grid-row: 2 / 4;  
}  
  
/* Items overlap - use z-index to control stacking */  
.item1 { z-index: 1; }  
.item2 { z-index: 2; }
```

Chapter 10

Advanced Positioning Techniques

10.1 Using justify-self for Horizontal Alignment

```
.container {  
  display: grid;  
  grid-template-columns: 200px 200px;  
  gap: 10px;  
}  
  
.item {  
  justify-self: center; /* Center horizontally in cell */  
  /* Options: start, end, center, stretch */  
}
```

10.2 Using align-self for Vertical Alignment

```
.container {  
  display: grid;  
  grid-template-rows: 100px 100px;  
  gap: 10px;  
}  
  
.item {  
  align-self: end; /* Align to bottom of cell */  
  /* Options: start, end, center, stretch */  
}
```

10.3 Combining Positioning with Alignment

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 100px);  
  grid-template-rows: repeat(3, 100px);  
  gap: 10px;  
}
```

```
.item:first-child {  
  grid-column: 1 / 3;  
  grid-row: 1 / 3;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}
```

10.4 Dynamic Positioning with CSS Variables

```
.container {  
  display: grid;  
  grid-template-columns: repeat(4, 100px);  
  --item-col: 1;  
  --item-row: 1;  
}  
  
.item {  
  grid-column: var(--item-col);  
  grid-row: var(--item-row);  
}  
  
.item:nth-child(2) {  
  --item-col: 2;  
  --item-row: 2;  
}
```

Chapter 11

Responsive Grid Positioning

11.1 Mobile-First Approach

```
HTML:
<div class="container">
  <header>Header</header>
  <main>Main</main>
  <aside>Aside</aside>
  <footer>Footer</footer>
</div>

CSS:
.container {
  display: grid;
  grid-template-columns: 1fr;
  grid-auto-rows: auto;
}

header { grid-column: 1; }
main { grid-column: 1; }
aside { grid-column: 1; }
footer { grid-column: 1; }

@media (min-width: 768px) {
  .container {
    grid-template-columns: 1fr 250px;
  }

  header { grid-column: 1 / -1; }
  footer { grid-column: 1 / -1; }
}
```

11.2 Desktop-First Approach

```
.container {
  display: grid;
  grid-template-columns: 1fr 250px;
```

```
    gap: 20px;
}

header { grid-column: 1 / -1; }
footer { grid-column: 1 / -1; }

@media (max-width: 768px) {
  .container {
    grid-template-columns: 1fr;
  }

  header { grid-column: 1; }
  footer { grid-column: 1; }
}
```

Chapter 12

Practice Exercises

12.1 Exercise 1: Create a 2x2 Grid

Create a grid with 2 columns and 2 rows. Position items so:

- Item 1 is in the top-left
- Item 2 is in the top-right
- Item 3 spans bottom row

Solution provided in supplementary materials

12.2 Exercise 2: Magazine Layout

Build a layout with:

- Full-width header
- 3-column main content area
- 1-column sidebar
- Full-width footer

12.3 Exercise 3: Dashboard

Create a dashboard where:

- One large widget spans 2x2
- Six smaller widgets fill remaining space
- All widgets have equal spacing

12.4 Exercise 4: Responsive Navigation

Build a navigation that:

- Displays as single column on mobile
- Becomes multi-column on tablet
- Horizontal layout on desktop

Chapter 13

Summary

13.1 Key Takeaways

- **Grid lines** are numbered starting from 1
- **grid-column** controls horizontal positioning
- **grid-row** controls vertical positioning
- **grid-area** combines both in one property
- **span** is often easier than calculating line numbers
- Use **auto-placement** by default, position only when needed
- Combine with **alignment properties** for fine-tuned control
- Make layouts **responsive** using media queries

13.2 Quick Reference

```
/* Column positioning */
grid-column: start / end;
grid-column: span num;

/* Row positioning */
grid-row: start / end;
grid-row: span num;

/* Combined */
grid-area: row-start / col-start / row-end / col-end;

/* Alignment */
justify-self: start | end | center | stretch;
align-self: start | end | center | stretch;

/* Auto-placement */
grid-auto-flow: row | column | row dense | column dense;
```

13.3 Next Steps

Now that you understand Grid Items Positioning, explore:

- CSS Grid Items Spanning - for advanced spanning techniques
- CSS Grid Lines Naming - for more readable positioning
- CSS Grid Area Naming - for template-based layouts
- Responsive Grid techniques